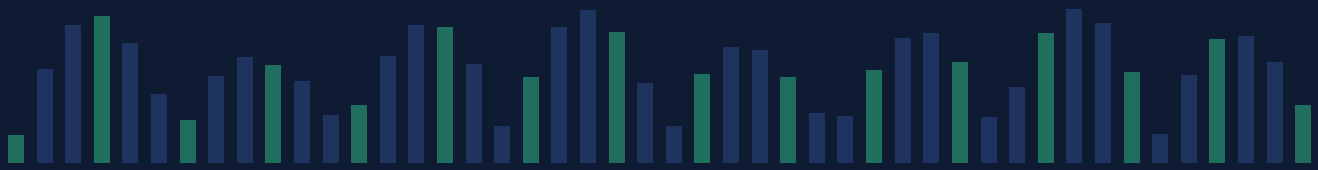
FINAL PRODUCT SPECIFICATION

Real-Time AI Interview Assistant

Product Features & Architecture

Every feature described in detail — what it does, how it works, the exact technology behind it, its performance target and its edge cases — plus the full system architecture, data model, APIs, deployment and release plan.

48	9	~0.5–0.85 s	4
Features specified	Product modules	First-word target (P50)	Release stages



Prepared for: Sourabh — BidYourStay

Date: October 2026 · Version 1.0 (final)

Builds on: Unified Low-Latency Blueprint (merge of Build Roadmap v2 + Architecture Plan)

Latency figures are engineering targets. Verify vendor features, model names and prices against live documentation before building.

CONTENTS

What's inside

#	Section	What you'll find
1	Product overview	What the product is, who it's for, the three usage modes
2	Feature catalogue at a glance	All 48 features with module, release stage and main technology
3	System architecture	Diagram, components, live-question sequence, data model, APIs, deployment
4	Technology stack	Every technology by layer and why it was chosen
5–13	Detailed features (modules A–I)	For each feature: what it does, how it works, technology, target, edge cases
14	Growth features	Career-platform features from the Build Roadmap, for later
15	Technology → feature matrix	Which technology powers which features
16	Performance targets	Latency budget and production SLOs
17	Release plan	Which features ship in MVP, v1, v1.1 and Growth
18	Risks & open checks	What to verify before and during the build

Release stage tags used throughout

Tag	Meaning	Roadmap phases
MVP	First usable product: live answers + screenshots	Phase 0–2 (~8–12 weeks)
v1	Sellable release: speed, privacy mode, reliability, billing	Phase 3–4 (+7–11 weeks)
v1.1	Practice mode, multi-language	Phase 5 / beta → GA
Growth	Career-platform features from Doc 1	Phase 6+

SECTION 1 · OVERVIEW

Product overview

A desktop app for Windows and macOS that listens to a live interview, detects each question, and shows a useful answer on a floating overlay in well under a second — plus one-hotkey solving of coding problems from screenshots.

Who it's for

- Job seekers in technical and non-technical interviews on Zoom, Google Meet, Microsoft Teams and Webex.
- Primary early market: India and the US (Indian accents, Hindi-English code-switching, INR pricing via Razorpay).
- Later: colleges and bootcamps via white-label (Growth).

Three usage modes

Mode	What the user does	Main features
Live Assist	Runs the app during a real call; answers appear automatically on the overlay.	B1–B6, C1–C8, E1–E6
Screenshot Solve	Presses a hotkey on a coding problem or diagram; a full solution streams in.	D1–D5
Practice	Talks to an AI interviewer; gets scores, a debrief and a readiness trend.	F1–F3

Product principles

- **Fastest first word wins.** Everything is streamed; nothing slow happens during the call that could happen before it.
- **Answer-first.** The first line is always the direct answer; detail follows.
- **Realtime is isolated.** The live path never shares resources with batch jobs (Doc 1).
- **Design for provider failure.** Every external AI service has a backup.
- **Desktop is untrusted.** No provider keys on the client — only short-lived tokens.
- **Honest claims.** Privacy-mode guarantees are stated per operating system, including the macOS 15+ limitation.
- **User controls the data.** Archiving is opt-in; retention and deletion are always available.

SECTION 2 · CATALOGUE

All features at a glance

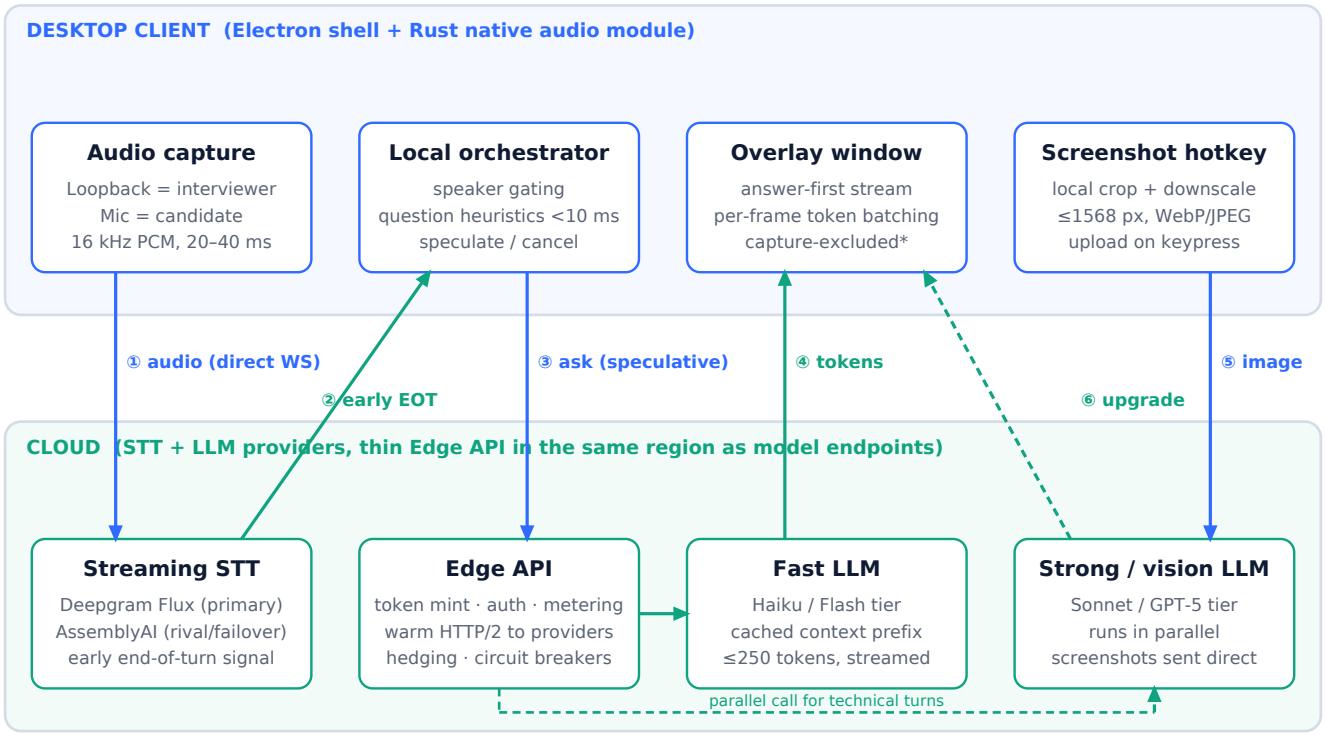
ID	Feature	Stage	Main technology
A	**Onboarding, Profile & Interview Setup**		
A1	Account & Sign-in	MVP	Supabase Auth (email OTP, Google OAuth), JWT + refresh tokens
A2	Resume Upload & AI Parsing	MVP	S3 / Cloudflare R2 (presigned upload), unpdf / pdf.js, mammoth
A3	Interview Session Setup	MVP	React + TypeScript main window, Fastify REST API
A4	Interview Pack (Context Precompute)	MVP	Claude Sonnet-tier model (generation), Redis (hot session state)
A5	Answer Style Preferences	MVP	React settings screen, Postgres (settings JSONB)
B	**Live Audio & Transcription**		
B1	System Audio Capture (Interviewer)	MVP	Rust native module (napi-rs), wasapi / cpal crates (Windows)
B2	Mic Capture & Speaker Separation	MVP	Same Rust module (cpal for mic input), Two STT streams or one multichannel stream
B3	Permissions & Audio Setup Wizard	MVP	Electron systemPreferences (media access), React wizard UI
B4	Streaming Transcription	MVP	Deepgram Flux (primary), AssemblyAI Universal-Streaming (secondary)
B5	Live Transcript Panel	MVP	React overlay renderer, Virtualised list
B6	Technical Vocabulary Boosting	v1	STT keyterm / custom-vocabulary features (verify support for the chosen model)
B7	Multi-language & Code-switching	v1.1	Multilingual STT models (Deepgram / AssemblyAI), LLM answer-language rules in the pack
C	**Question Detection & Answer Engine**		
C1	Question Detection	MVP	TypeScript local orchestrator (Electron main), Rule-based classifier; optional tiny local model later
C2	Speculative Answering	v1	Deepgram Flux early end-of-turn events, AbortController + Edge API stream cancel
C3	Question Classification & Routing	MVP	TypeScript rules in the local orchestrator
C4	Instant Answer (Fast Path)	MVP	Claude Haiku-tier model (current: Haiku 5.5) or Gemini Flash-tier alternative, Fastify streaming proxy
C5	Deep Answer Upgrade	v1	Claude Sonnet-tier model (current: Sonnet 5.5) or GPT-5-tier alternative, Claude Opus tier (current: Opus 5.5) for the hardest problems
C6	Conversation Memory & Follow-ups	MVP	Redis session state, Local in-memory cache
C7	Manual Controls	MVP	Electron globalShortcut, IPC between main and overlay
C8	Answer Templates by Question Type	MVP	Versioned prompt templates in the repo, Template version logged in telemetry for A/B tests
D	**Screenshot & Coding Solver**		
D1	Screenshot Capture	MVP	Electron desktopCapturer or a native capture crate in the Rust module
D2	Image Preprocessing	MVP	sharp (libvips) in Electron main, or Rust image crate

ID	Feature	Stage	Main technology
D3	Coding Problem Solver	MVP	Claude Sonnet-tier vision (default), Claude Opus tier for hardest problems
D4	Multi-Screenshot Context	v1	Multi-image model requests
D5	Code Follow-ups	MVP	Same model routing as D3
E	**Overlay & Desktop Experience**		
E1	Overlay Window	MVP	Electron BrowserWindow, React + TypeScript
E2	Streaming Renderer	MVP	requestAnimationFrame batching, markdown-it (or similar)
E3	Global Hotkeys	MVP	Electron globalShortcut, Settings stored locally and in Postgres
E4	Screen-Share Privacy Mode	v1	Electron setContentProtection, Win32 SetWindowDisplayAffinity
E5	Layout Modes & Multi-monitor	v1	Electron screen API, React layout components
E6	Session Controls & Status	MVP	React status bar, WebSocket heartbeat
F	**Practice Mode (Mock Interviews)**		
F1	AI Mock Interviewer	v1.1	Claude Sonnet-tier model (question plan, follow-ups), Streaming TTS (bake-off: Cartesia, ElevenLabs, OpenAI TTS)
F2	Scoring & Debrief	v1.1	Claude Sonnet-tier model, BullMQ worker
F3	Readiness Score & Trends	v1.1	Postgres, Recharts in the main window
G	**Session History & Data Control**		
G1	Session Archive (opt-in)	v1	Postgres full-text search (tsvector), S3 / R2 with encryption at rest
G2	Post-Interview Summary	v1	Claude Sonnet-tier model, BullMQ worker
G3	Privacy & Retention Controls	MVP	BullMQ scheduled jobs, Postgres
H	**Accounts, Billing & Admin**		
H1	Plans & Credits	v1	Stripe Billing (global), Razorpay Subscriptions (India)
H2	Usage Metering	MVP	Redis counters, Postgres usage_events
H3	Admin Console	v1	Next.js admin app, Postgres
H4	Abuse & Fraud Protection	v1	@fastify/rate-limit with Redis, Stripe Radar
I	**Platform & Reliability**		
I1	Provider Failover & Circuit Breakers	v1	opossum (Node circuit breaker), Edge API router
I2	Hedged Requests	v1	Fastify LLM proxy, AbortController
I3	Observability	MVP	OpenTelemetry (Node + Electron), Prometheus + Grafana
I4	Auto-Update & Distribution	MVP	electron-builder, electron-updater
I5	Security	MVP	Electron security settings, safeStorage
I6	Compatibility Test Harness	v1	Scripted test VMs / devices, Playwright for browser-based meeting clients
I7	Latency & Accuracy Bake-off Harness	MVP	Node / Python scripts, Stored audio corpus with reference transcripts

SECTION 3 · ARCHITECTURE

System architecture

Four layers: the desktop client, a thin Edge API, background workers, and external AI providers. The live path is client → STT provider directly, and client → Edge API → LLM provider, streaming at every hop.



** capture exclusion is reliable on Windows 10 2004+/11 and macOS ≤14 only — see Section 8*

3.1 Architecture principles

- **Stream everything** — audio, transcripts and answer tokens flow continuously; no stage waits for a full result.
- **Precompute context** — the Interview Pack is built before the call and kept hot in Redis and the provider's prompt cache.
- **Thin, stateless Edge API** — auth, token minting, routing, metering, failover; never buffers a response.
- **Measure every hop** — one trace per answer with per-stage timings.

3.2 Components

Component	Runs where	Responsibilities	Technology
Rust audio module	Desktop (native)	Loopback + mic capture, framing, resampling, screenshot capture (optional)	Rust, napi-rs, wasapi/cpal, ScreenCaptureKit helper, rubato
Electron main process	Desktop	Local orchestrator (question detection, routing, speculation), STT WebSocket client, Edge API WebSocket, hotkeys, updater, secure storage	Electron, TypeScript, ws, globalShortcut, electron-updater, safeStorage
Main window	Desktop (renderer)	Sign-in, resume, session setup, settings, history, practice mode, billing links	React, TypeScript, Tailwind, Recharts
Overlay window	Desktop (renderer)	Answer panel, transcript strip, status bar, screenshot selection layer	React, rAF token batching, markdown-it, highlight.js
Edge API	Cloud (2–3 regions)	JWT auth, STT token minting, session WebSocket, LLM proxy + router, hedging, circuit breakers, metering, rate limits, billing webhooks	Node.js, Fastify, @fastify/websocket, opossum, OpenTelemetry
Workers	Cloud	Resume parsing, Interview Pack generation, summaries, mock-interview scoring, retention jobs, usage roll-ups	BullMQ (Redis), Node.js
Data stores	Cloud	Users, profiles, sessions, turns, usage; hot session state; files	Postgres (Supabase/Neon), Redis, S3/R2
External providers	Vendor cloud	Speech-to-text, LLM answers, vision, TTS (practice)	Deepgram, AssemblyAI, Anthropic, OpenAI, Google, TTS vendor
Ops	Cloud	Dashboards, alerts, error tracking, CI/CD	Prometheus, Grafana, Sentry, GitHub Actions

3.3 Sequence — one live question

t (ms)	Step	Where
0	Interviewer finishes speaking	Meeting app
0–60	Last audio frames captured and streamed to STT over a direct WebSocket	Rust module → STT
~150–250	STT emits early end-of-turn; question detected in <10 ms; transcript strip highlights the question	STT → orchestrator → overlay
~160–280	Answer request sent: cached pack prefix + recent turns + question; deep model fired in parallel if technical	Orchestrator → Edge API → LLM
~500–850	First answer token arrives and is painted on the next animation frame	LLM → Edge API → overlay
~1,000–2,000	Deep answer (technical turns) starts streaming below the fast answer	Strong LLM → overlay
if speech resumes	Request aborted and re-sent with the longer transcript (max 2 restarts)	Orchestrator
end of turn	Turn + answer summary saved to memory; usage counters updated	Redis / Postgres

3.4 Data model (core tables)

Table	Key fields	Notes
users	id, email, auth_provider, plan, region, created_at	Managed by Supabase Auth + app table
devices	id, user_id, os, os_version, app_version, last_seen	Device limits, compatibility analytics
resumes	id, user_id, file_url, parsed_json, confidence_json, version	Several per user
profiles	user_id, skills[], stories_json, preferences_json	Normalised skills, STAR stories, answer style
interview_sessions	id, user_id, resume_id, company, role, round_type, jd_text, jd_analysis_json, language, status, started_at, ended_at	One per interview
interview_packs	session_id, pack_text, token_count, version, built_at	Cached prefix source
turns	id, session_id, speaker, text, question_type, t_start, t_end	Stored only if archiving is on
answers	id, turn_id, model, tier (fast/deep), text, ttft_ms, tokens_in, tokens_out, cancelled	Latency + cost per answer
screenshots	id, session_id, file_url, width, height, model, ttft_ms	Files in S3/R2
practice_results	id, session_id, scores_json, debrief_json	Practice mode
usage_events	id, user_id, session_id, kind (stt_min / tok_in / tok_out / image), qty, cost_usd	Billing + margins
subscriptions	user_id, provider (stripe/razorpay), plan, status, period_end	Webhook-driven

Redis keys (hot state): `session:{id}:pack`, `session:{id}:turns`, `session:{id}:usage`, `ratelimit:{user}`, `breaker:{provider}`.

3.5 APIs and realtime messages

Endpoint / message	Purpose
`POST /v1/resumes` · `GET /v1/resumes/:id`	Upload (presigned URL) and read parsed resume
`POST /v1/sessions` · `PATCH /v1/sessions/:id`	Create / edit interview session; triggers JD analysis and pack build
`POST /v1/sessions/:id/start`	Loads pack into Redis, warms LLM connection, returns STT token + session WS URL
`POST /v1/stt-token`	Mints a fresh short-lived STT token (renewal / failover)
`WS /v1/live/:sessionId`	Client ↔ Edge API realtime channel
→ `ask {turnId, text, type, deep}`	Request an answer (speculative or final)
→ `cancel {turnId}`	Abort a speculative answer
→ `image {turnId, blobRef}`	Screenshot solve request
← `token {turnId, tier, delta}`	Streamed answer tokens (fast or deep)
← `done {turnId, usage}` · `error {code}` · `status {...}`	Completion, errors, provider health
`POST /v1/sessions/:id/end`	Flush usage, schedule summary / retention jobs
`POST /webhooks/stripe` · `/webhooks/razorpay`	Billing events

3.6 Deployment & infrastructure

Area	Choice	Detail
Edge API hosting	Fly.io or AWS Fargate	2–3 regions near model endpoints (us-east, eu-west); test an India region and cloud-hosted model endpoints (AWS Bedrock / Google Vertex) for Indian users.
Front door	Cloudflare	TLS, DDoS protection, WebSocket proxying.
Database	Postgres on Supabase or Neon	Managed backups; read replica later.
Cache / queues	Redis (Upstash or ElastiCache)	Hot session state, rate limits, BullMQ queues.
Files	S3 or Cloudflare R2	Encrypted at rest; lifecycle rules for retention.
CI/CD	GitHub Actions	Build, sign and notarise installers; deploy API with canary; run bake-off and compatibility jobs.
Desktop distribution	electron-builder + electron-updater	Windows installer, macOS DMG; staged rollouts.
Monitoring	OpenTelemetry → Prometheus/Grafana; Sentry	Per-stage latency, provider health, crashes.

3.7 Security architecture

- Provider API keys live only in the Edge API's secret store; the client receives short-lived STT tokens and its own user JWT.
- Electron hardening: context isolation, sandboxed renderers, no Node.js in renderer windows, strict content-security policy.
- Refresh tokens encrypted with OS facilities (safeStorage → Keychain / DPAPI).
- TLS on every connection; database and file storage encrypted at rest; per-user access checks on every request.
- Audio is never stored by default; transcripts only with opt-in archiving.

SECTION 4 · STACK

Technology stack by layer

Layer	Technology	Why
Desktop shell	Electron + React + TypeScript	Working capture-exclusion API on both OSes; fastest path to cross-platform overlay. Port to Tauri later if needed.
Native audio	Rust via napi-rs (wasapi/cpal, ScreenCaptureKit, rubato)	Low-latency capture; reusable in Tauri later.
Speech-to-text	Deepgram Flux (primary), AssemblyAI Universal-Streaming (backup)	Built-in early / final end-of-turn signals; decide by bake-off.
Fast LLM	Claude Haiku tier (current: Haiku 5.5) / Gemini Flash tier	Lowest time-to-first-token with good quality.
Strong / vision LLM	Claude Sonnet tier (Sonnet 5.5), Opus tier (Opus 5.5) for hardest; GPT-5 tier alt	Accuracy on coding, design and screenshots.
TTS (practice)	Cartesia / ElevenLabs / OpenAI TTS (bake-off)	Streaming speech for the AI interviewer.
Edge API	Node.js + Fastify, WebSocket, opossum	Fast streaming proxy; one language with the desktop app.
Workers	BullMQ	Simple Redis-based jobs; no Kafka needed early.
Data	Postgres (Supabase/Neon), Redis, S3/R2	Managed, cheap, easy auth integration.

Layer	Technology	Why
Auth	Supabase Auth	Email OTP + Google OAuth out of the box.
Billing	Stripe + Razorpay	Global + India.
Observability	OpenTelemetry, Prometheus, Grafana, Sentry	Per-stage latency from day one.
CI/CD	GitHub Actions, electron-builder	Signed builds and staged rollout.

SECTION 5 · MODULE A

A. Onboarding, Profile & Interview Setup

Everything that happens before the interview. The goal is to prepare all context in advance so nothing slow happens during the live session.

A1 Account & Sign-in	MVP
What it does	Users create an account with an email one-time code or Google. The desktop app signs in through the system browser and receives the session through a deep link, so passwords are never typed into the overlay app.
How it works	1. Desktop app opens the web login page in the default browser. 2. Supabase Auth issues an access token (short-lived) and a refresh token. 3. Browser redirects to a custom protocol link (e.g. `yourapp://auth`); Electron receives the tokens. 4. Refresh token is encrypted with Electron `safeStorage` (OS keychain / DPAPI). 5. Every Edge API request carries the access token; Fastify verifies the JWT.
Technology	Supabase Auth (email OTP, Google OAuth) · JWT + refresh tokens · Electron protocol handler · Electron safeStorage · Fastify + @fastify/jwt
Target	Sign-in under 30 s; silent token refresh.
Edge cases	Offline start shows cached profile but blocks live sessions. Device limit per plan (e.g. 2 devices).

A2 Resume Upload & AI Parsing	MVP
What it does	The user uploads a PDF or DOCX resume. AI turns it into a structured profile: roles, projects, measurable results, skills with years of experience, education and links.
How it works	1. File uploads straight to object storage via a presigned URL. 2. Worker extracts text (PDF: unpdf / pdf.js; DOCX: mammoth). 3. Fast LLM returns strict JSON against a schema, with a confidence score per field (from Doc 1). 4. Skill names are normalised (React = ReactJS) and saved as a skills list. 5. User reviews and edits the result in the main window.
Technology	S3 / Cloudflare R2 (presigned upload) · unpdf / pdf.js, mammoth · Claude Haiku-tier model with structured JSON output · BullMQ worker · Postgres (JSONB profile)
Target	Parse in under 20 s; over 80% field accuracy.
Edge cases	Scanned PDFs have no text layer: fall back to a vision model on page images. Users can keep several resumes.

A3 Interview Session Setup	MVP
What it does	Before each interview the user enters company, role, round type (HR, behavioural, technical, coding, system design), interview date/time, answer language and pastes the job description.
How it works	1. Simple form in the main window, prefilled from the last session. 2. JD Analyzer (from Doc 1) splits required vs preferred skills, key responsibilities and likely question topics with weights. 3. Session is saved as a draft and linked to the chosen resume.
Technology	React + TypeScript main window · Fastify REST API · Haiku-tier model for JD analysis · Postgres
Target	Setup under 2 minutes; JD analysis under 10 s.
Edge cases	No JD available: build the session from company + role only and mark context as 'thin'.

A4 Interview Pack (Context Precompute)	MVP
What it does	The single most important latency feature before the call. A compact 1,000–2,000-token context pack is built ahead of time so every live prompt is short, identical at the start, and cacheable.
How it works	1. Generated when the session is created, using a stronger model (quality matters here, speed does not). 2. Contents: 5-line candidate summary, 5–8 STAR stories, top skills with years, JD must-haves, company notes, answer-style rules. 3. Stored in Postgres; a hot copy is loaded into Redis when the session starts. 4. Placed as the fixed prefix of every live prompt so the provider's prompt cache can reuse it. 5. A warm-up request at session start primes the cache and the connection.
Technology	Claude Sonnet-tier model (generation) · Redis (hot session state) · Provider prompt caching · Postgres
Target	Pack ready within 60 s of setup; cache hit rate over 90% during a session.
Edge cases	If the user edits the pack mid-session the cache is rebuilt once; keep the pack under the size limit.

A5 Answer Style Preferences	MVP
What it does	Users control how answers look: short or medium length, talking points vs first-person script, tone, answer language and preferred coding language.
How it works	1. Settings saved per user with per-session overrides. 2. Compiled into the rules section of the Interview Pack so they cost nothing at runtime.
Technology	React settings screen · Postgres (settings JSONB) · Versioned prompt templates
Target	Changes apply to the next answer instantly.
Edge cases	Conflicting settings (e.g. 'short' + 'detailed code') resolve by question type.

SECTION 6 · MODULE B

B. Live Audio & Transcription

Capturing the interviewer's voice and turning it into text with the lowest possible delay.

B1 System Audio Capture (Interviewer)	MVP
What it does	Hears the interviewer through the meeting app's audio output (Zoom, Meet, Teams, Webex) without installing a virtual audio driver.
How it works	1. Windows: WASAPI loopback on the active output device. 2. macOS 13+: ScreenCaptureKit audio stream (needs Screen Recording permission). 3. Audio is cut into 20–40 ms frames and resampled to 16 kHz mono 16-bit PCM inside Rust (microseconds of work). 4. Frames are handed to the Electron main process through a napi-rs binding.
Technology	Rust native module (napi-rs) · wasapi / cpal crates (Windows) · ScreenCaptureKit via Swift helper or Rust bindings (macOS) · rubato resampler crate
Target	Capture-to-send under 40 ms; under 3% CPU.
Edge cases	Output device changes (headphones plugged in) trigger automatic re-binding. macOS 12 and older need a virtual audio device and are out of scope for v1.

B2 Mic Capture & Speaker Separation	MVP
What it does	Captures the candidate's microphone as a separate channel so the system always knows who is speaking and never answers the candidate's own voice.
How it works	1. Loopback channel is labelled 'interviewer'; mic channel is labelled 'candidate'. 2. Only interviewer turns can trigger answers. 3. If the user is on speakers (no headphones), a cross-channel energy check ignores interviewer echo picked up by the mic.
Technology	Same Rust module (cpal for mic input) · Two STT streams or one multichannel stream
Target	Answers triggered by the candidate's own speech: under 2%.
Edge cases	Recommend headphones in the setup wizard for best accuracy.

B3 Permissions & Audio Setup Wizard	MVP
What it does	A first-run checklist that gets microphone and screen-recording permissions, picks devices, shows level meters and runs a 10-second test transcription.
How it works	1. Checks permission status and opens the exact OS settings page when missing. 2. Plays a test clip and confirms the loopback captures it. 3. Runs a short live transcription to confirm the STT connection.
Technology	Electron systemPreferences (media access) · React wizard UI · Rust module test hooks
Target	Setup completed in under 3 minutes.
Edge cases	macOS requires an app restart after granting Screen Recording; the wizard explains this.

B4 Streaming Transcription	MVP
What it does	Real-time interviewer text with partial results while they speak, final results, and turn events.
How it works	1. Edge API mints a short-lived STT token for the session. 2. Electron main opens a WebSocket directly to the STT provider (audio never passes through your servers). 3. Partials, finals and early / final end-of-turn events flow to the local orchestrator. 4. On error or disconnect, the client switches to the secondary provider and replays up to 5 s of buffered audio.
Technology	Deepgram Flux (primary) · AssemblyAI Universal-Streaming (secondary) · WebSocket client (ws) in Electron main · Edge API token minting
Target	Partial transcript latency at or under 300 ms.
Edge cases	Network drop: keep buffering locally, reconnect with backoff, show a status warning.

B5 Live Transcript Panel	MVP
What it does	Optional strip in the overlay that shows the last few lines with speaker labels and highlights detected questions.
How it works	1. Shows partial text in grey, final text in normal colour. 2. Detected question is highlighted the moment it is detected, giving instant feedback before the answer starts.
Technology	React overlay renderer · Virtualised list
Target	Updates within one animation frame of arrival.
Edge cases	Can be hidden for a minimal overlay.

B6 Technical Vocabulary Boosting	v1
What it does	Improves recognition of technical words and names such as Kubernetes, PostgreSQL, the company's product names and the candidate's own project names.
How it works	1. Keyterm list is built automatically from the Interview Pack (JD skills, resume technologies, company name). 2. Sent to the STT provider as keyterms / custom vocabulary where the chosen model supports it.
Technology	STT keyterm / custom-vocabulary features (verify support for the chosen model)
Target	Measurably lower error rate on technical terms vs baseline in the bake-off.
Edge cases	Too many keyterms can hurt accuracy; cap the list (e.g. top 50).

B7 Multi-language & Code-switching	v1.1
What it does	Support for interviews in other languages and Hindi-English code-switching common in Indian interviews.
How it works	1. Session language set in A3; multilingual STT model selected automatically. 2. Answer language follows the user setting, independent of the interview language.
Technology	Multilingual STT models (Deepgram / AssemblyAI) LLM answer-language rules in the pack
Target	Same latency targets as English.
Edge cases	Benchmark accuracy per language before advertising it.

SECTION 7 · MODULE C

C. Question Detection & Answer Engine

The core of the product: decide that a question was asked, and put a useful answer on screen as fast as possible.

C1 Question Detection	MVP
What it does	Decides, in under 10 ms, that the interviewer has asked something worth answering.
How it works	1. Inputs: STT early end-of-turn signal and confidence, final end-of-turn, speaker channel. 2. Interrogative heuristics: wh-words, 'can you', 'tell me about', 'walk me through', 'how would you', trailing '?'. 3. Ignores fillers and acknowledgements ('okay', 'great', 'makes sense'). 4. 400–600 ms silence floor used to confirm (not to start) the answer.
Technology	TypeScript local orchestrator (Electron main) Rule-based classifier; optional tiny local model later
Target	Decision under 10 ms; false triggers at most 5%; missed questions at most 5%.
Edge cases	Multi-part questions are merged when the second part arrives within the speculation window.

C2 Speculative Answering	v1
What it does	Starts the answer before the interviewer has fully finished, which is the biggest single latency saving in the system.
How it works	1. On the STT's early end-of-turn signal, the orchestrator sends the answer request immediately. 2. If the interviewer keeps talking, the request is aborted and re-sent with the longer transcript. 3. Maximum 2 restarts per turn to control cost; waste rate is tracked.
Technology	Deepgram Flux early end-of-turn events AbortController + Edge API stream cancel
Target	Saves roughly 200–500 ms; cancelled-call waste at most 30%.
Edge cases	Tune the early-turn threshold per round type (HR talk is slower than coding questions).

C3 Question Classification & Routing	MVP
What it does	Labels each question so the right prompt template and model are used: behavioural, technical concept, coding, system design, HR / salary, small talk.
How it works	1. Keyword rules (e.g. 'complexity', 'design a system', 'conflict', 'expected salary'). 2. Decides the template (C8) and whether to also fire the strong model (C5).
Technology	TypeScript rules in the local orchestrator
Target	Under 5 ms.
Edge cases	Unknown type falls back to the general template with fast model only.

C4 Instant Answer (Fast Path)	MVP
What it does	The first answer on screen, aimed at about 0.5–0.85 s after the interviewer stops speaking.
How it works	1. Prompt = cached Interview Pack prefix + last few turns + the question. 2. Edge API forwards the request over a warm HTTP/2 connection and streams tokens back over the client WebSocket without buffering. 3. Answer-first format: one-line direct answer, then 2–4 bullets, capped at about 250 tokens.
Technology	Claude Haiku-tier model (current: Haiku 5.5) or Gemini Flash-tier alternative; Fastify streaming proxy; Provider prompt caching; Hedged requests (I2)
Target	First word P50 at or under 0.8 s; P95 at or under 1.5 s.
Edge cases	If no model responds, show the transcript and a 'retry' button (graceful degradation from Doc 1).

C5 Deep Answer Upgrade	v1
What it does	A more accurate answer for technical, coding and system-design questions, without making the user wait for it.
How it works	1. Fired in parallel with C4 when the route is technical. 2. Streams into a 'Deeper answer' section under the fast answer; user can switch views with a hotkey.
Technology	Claude Sonnet-tier model (current: Sonnet 5.5) or GPT-5-tier alternative; Claude Opus tier (current: Opus 5.5) for the hardest problems
Target	First token about 1–2 s, while the fast answer is already visible.
Edge cases	Skip the deep call on short follow-ups to save cost.

C6 Conversation Memory & Follow-ups	MVP
What it does	Understands follow-ups like 'why did you choose that?' or 'what would you do differently?' by remembering recent turns.
How it works	1. Keeps the last 6–10 turns (question + short answer summary) in local memory and Redis. 2. Placed after the cached prefix so caching still works. 3. In long sessions, a background job writes a rolling summary every ~10 turns.
Technology	Redis session state; Local in-memory cache; Fast model for background summaries
Target	Adds no measurable latency to the live path.
Edge cases	Interviewer changes topic: memory is kept, but older turns are summarised.

C7 Manual Controls	MVP
What it does	The user can always take control: answer now, regenerate, shorter, more detail, copy, or type a question manually.
How it works	1. Global hotkeys and overlay buttons call the same answer pipeline with a modifier. 2. Typed questions skip STT and go straight to C3/C4.
Technology	Electron globalShortcut · IPC between main and overlay
Target	Same latency as automatic answers.
Edge cases	Hotkey conflicts with the meeting app are detected and the user is asked to remap.

C8 Answer Templates by Question Type	MVP
What it does	Each question type gets a structure that is easy to read aloud under pressure.
How it works	1. Behavioural: STAR (Situation, Task, Action, Result) using a matching story from the pack. 2. Technical concept: Point / Evidence / Trade-off (from Doc 1). 3. Coding: approach, code, time/space complexity, edge cases. 4. System design: requirements, high-level design, data model, scaling, trade-offs. 5. HR / salary: framing that uses the user's own expectations and notice period.
Technology	Versioned prompt templates in the repo · Template version logged in telemetry for A/B tests
Target	Templates add no latency (part of cached prefix).
Edge cases	Templates are tuned using real (opt-in) transcripts during beta.

SECTION 8 · MODULE D

D. Screenshot & Coding Solver

Solve what is on screen — coding problems, diagrams, take-home prompts — with one hotkey.

D1 Screenshot Capture	MVP
What it does	A hotkey captures a selected region, a window or the full screen.
How it works	1. Hotkey opens a transparent selection layer. 2. Capture runs natively and the selected region is cropped locally.
Technology	Electron desktopCapturer or a native capture crate in the Rust module
Target	Capture under 50 ms.
Edge cases	Multi-monitor and high-DPI scaling handled per display.

D2 Image Preprocessing	MVP
What it does	Makes the image small enough to upload fast while keeping code perfectly readable.
How it works	1. Downscale the long edge to at most 1568 px, keep text sharp. 2. Encode as WebP or JPEG (quality ~85). 3. Upload starts the moment encoding finishes.
Technology	sharp (libvips) in Electron main, or Rust image crate
Target	Under 60 ms; typical payload 100–300 KB.
Edge cases	Very small fonts: keep higher resolution and accept a slightly larger upload.

D3 Coding Problem Solver	MVP
What it does	Reads the problem from the screenshot and returns the approach, working code in the chosen language, complexity, edge cases and a short dry run.
How it works	1. Image + cached pack prefix + language preference sent directly to the vision model (no OCR step). 2. Answer streams into the overlay with code highlighting. 3. Optional: fast model gives a one-line approach in parallel while the full solution streams.
Technology	Claude Sonnet-tier vision (default) · Claude Opus tier for hardest problems · GPT-5-tier alternative
Target	First word about 0.7–1.6 s.
Edge cases	Blurry or partial image: model is told to ask for another screenshot rather than guess.

D4 Multi-Screenshot Context	v1
What it does	Long problems that need scrolling can be captured as 2–4 screenshots and sent together.
How it works	1. 'Add to context' hotkey stacks images; 'Solve' sends them in one request.
Technology	Multi-image model requests
Target	Same as D3 plus upload time per image.
Edge cases	Cap images per request to control cost and latency.

D5 Code Follow-ups	MVP
What it does	Follow-up actions on a solution: optimise, explain a line, add comments, convert language, dry-run with an example.
How it works	1. Uses conversation memory (C6) with the last solution attached.
Technology	Same model routing as D3
Target	Fast-path latency for short follow-ups.

SECTION 9 · MODULE E

E. Overlay & Desktop Experience

The window the user actually looks at during the interview.

E1 Overlay Window	MVP
What it does	A floating answer panel that stays on top of the meeting app.
How it works	1. Separate Electron BrowserWindow: transparent, frameless, always-on-top, hidden from the taskbar. 2. Click-through mode via setIgnoreMouseEvents, toggled with a hotkey. 3. Draggable, resizable, adjustable opacity and font size.
Technology	Electron BrowserWindow · React + TypeScript · Tailwind CSS
Target	Opens in under 200 ms; idle CPU near zero.
Edge cases	Remembers position and size per monitor setup.

E2 Streaming Renderer	MVP
What it does	Renders tokens smoothly even at 150+ tokens per second.
How it works	1. Tokens collect in a buffer and render once per animation frame. 2. Incremental markdown; code is highlighted when a code block closes.
Technology	requestAnimationFrame batching · markdown-it (or similar) · highlight.js / Prism
Target	Under 16 ms per frame, no visible jank.
Edge cases	Very long answers auto-scroll unless the user scrolls up.

E3 Global Hotkeys	MVP
What it does	Keyboard control without touching the mouse. Defaults (all remappable): show/hide overlay, answer now, screenshot, add screenshot to context, scroll up/down, regenerate, shorter / more detail, toggle click-through, switch fast/deep answer, clear.
How it works	1. Registered globally so they work while the meeting app has focus.
Technology	Electron globalShortcut · Settings stored locally and in Postgres
Target	Hotkey to action under 20 ms.
Edge cases	Detect conflicts with OS and meeting-app shortcuts.

E4 Screen-Share Privacy Mode	v1
What it does	Asks the operating system to exclude the overlay from screen capture, with an honest per-OS status indicator and a self-test.
How it works	1. Windows 10 2004+ / 11: SetWindowDisplayAffinity with WDA_EXCLUDEFROMCAPTURE (Electron setContentProtection). 2. macOS: NSWindow.sharingType = none (same Electron call). 3. Status badge shows Strong (Windows, macOS 14 and older) or Reduced (macOS 15+). 4. 'Test my setup' takes a local screen capture and checks whether the overlay appears in it.
Technology	Electron setContentProtection · Win32 SetWindowDisplayAffinity · AppKit NSWindow.sharingType · desktopCapturer (self-test)
Target	Applied before the overlay is first shown.
Edge cases	Not reliable on macOS 15+ against ScreenCaptureKit-based sharing. Never hides from cameras, process lists or proctoring tools — state this in the product.

E5 Layout Modes & Multi-monitor	v1
What it does	Compact mode (answer only) and expanded mode (transcript + fast answer + deep answer tabs); snap to screen edges; per-display positions.
How it works	1. Layout switches with a hotkey; positions saved per display arrangement.
Technology	Electron screen API · React layout components
Target	Instant switch.
Edge cases	Display unplugged mid-session: overlay moves to the primary screen.

E6 Session Controls & Status	MVP
What it does	Start, pause, mute capture and end the session; live status dots for audio, STT and AI; minutes used vs plan.
How it works	1. WebSocket heartbeat every few seconds; automatic reconnect with backoff. 2. Usage counter updates from the Edge API metering.
Technology	React status bar · WebSocket heartbeat · Edge API metering (H2)
Target	Disconnect noticed within 3 s.
Edge cases	Plan limit reached: warn at 5 minutes left, never cut a session mid-answer.

SECTION 10 · MODULE F

F. Practice Mode (Mock Interviews)

Reuses the same audio → STT → LLM pipeline for practice. Lower risk, good for trust and retention (ideas from Doc 1, Phase 2).

F1 AI Mock Interviewer	v1.1
What it does	An AI interviewer asks questions by voice based on the JD and round type; the user answers aloud.
How it works	1. Question plan generated from the Interview Pack. 2. Questions are spoken with streaming text-to-speech; user answers through the mic. 3. Follow-up questions adapt to what the user said.
Technology	Claude Sonnet-tier model (question plan, follow-ups) · Streaming TTS (bake-off: Cartesia, ElevenLabs, OpenAI TTS) · Same STT pipeline (B4)
Target	AI starts speaking within about 1 s of the user finishing.
Edge cases	Barge-in: if the user starts talking, the AI stops speaking.

F2 Scoring & Debrief	v1.1
What it does	After each mock: scores for technical accuracy, clarity, STAR completeness, filler words and speaking pace, with a better example answer per question.
How it works	1. Pace (words per minute) and fillers computed from transcript timestamps. 2. Evaluation runs as a background job and appears within a minute.
Technology	Claude Sonnet-tier model · BullMQ worker · Postgres
Target	Debrief ready within 60 s.

F3 Readiness Score & Trends	v1.1
What it does	Technical, Communication, System Design and Behavioural percentages (from Doc 1) with a trend chart across sessions.
How it works	1. Scores stored per session and plotted over time.
Technology	Postgres · Recharts in the main window
Target	Chart loads under 500 ms.

SECTION 11 · MODULE G

G. Session History & Data Control

What happens after the interview, and who controls the data.

G1 Session Archive (opt-in)	v1
What it does	Saved transcript, detected questions, answers and screenshots for each session, searchable.
How it works	1. Saved only if the user turns archiving on. 2. Full-text search across all sessions.
Technology	Postgres full-text search (tsvector) · S3 / R2 with encryption at rest
Target	Search results under 300 ms.

G2 Post-Interview Summary	v1
What it does	A summary of questions asked, topics covered, weak spots to study, and a draft thank-you email.
How it works	1. Background job runs when the session ends.
Technology	Claude Sonnet-tier model · BullMQ worker
Target	Ready within 2 minutes of session end.
Edge cases	Only available when archiving is on.

G3 Privacy & Retention Controls	MVP
What it does	Users choose retention (none / 7 / 30 / 90 days), can delete everything at any time, and can use a local-only mode where no transcript is stored on the server (only usage counts for billing).
How it works	1. Scheduled deletion jobs; storage lifecycle rules as a second safety net.
Technology	BullMQ scheduled jobs · Postgres · S3 / R2 lifecycle rules
Target	Deletion completes within 24 h.
Edge cases	Provider-side retention depends on each AI vendor's API data policy — document it.

SECTION 12 · MODULE H

H. Accounts, Billing & Admin

How the product makes money and how the team runs it.

H1 Plans & Credits	v1
What it does	Free trial minutes, monthly and annual plans, and minute packs; local pricing for India.
How it works	1. Checkout in the browser; webhooks update the plan in Postgres; the app reads entitlements on start.
Technology	Stripe Billing (global) · Razorpay Subscriptions (India) · Fastify webhook handlers
Target	Plan active within 10 s of payment.
Edge cases	Failed renewals: grace period, then downgrade.

H2 Usage Metering	MVP
What it does	Counts STT minutes, LLM tokens in/out and screenshots per session, and calculates cost per session.
How it works	1. Edge API increments counters in Redis during the session; flushed to Postgres at session end; nightly roll-ups.
Technology	Redis counters · Postgres usage_events · Cron / BullMQ roll-ups
Target	Zero added latency on the live path.

H3 Admin Console	v1
What it does	Internal dashboard: users, sessions, revenue, cost per session, latency P50/P95, provider health, refunds.
How it works	1. Read-only views over Postgres plus embedded Grafana panels.
Technology	Next.js admin app · Postgres · Grafana
Target	—
Edge cases	Role-based access for team members.

H4 Abuse & Fraud Protection	v1
What it does	Rate limits per user and IP, device limits, one live session per account, payment fraud screening.
How it works	1. Enforced in the Edge API before any provider call.
Technology	@fastify/rate-limit with Redis · Stripe Radar
Target	—
Edge cases	Account sharing detection via device fingerprints.

SECTION 13 · MODULE I

I. Platform & Reliability

Features users never see but always feel. Most come from Doc 1's reliability work.

I1 Provider Failover & Circuit Breakers	v1
What it does	If an AI or STT provider slows down or fails, traffic moves to the backup automatically.
How it works	1. Per-provider circuit breaker; half-open probe every 30 s (Doc 1). 2. STT: Deepgram ↔ AssemblyAI. LLM: Anthropic ↔ OpenAI / Google.
Technology	opossum (Node circuit breaker) · Edge API router
Target	Failover under 200 ms; alert within 60 s of an open circuit.

I2 Hedged Requests	v1
What it does	If no first token arrives within ~700 ms, the same request is sent to the backup provider and whichever streams first wins.
How it works	1. Implemented in the Edge API's LLM proxy; the slower stream is cancelled.
Technology	Fastify LLM proxy · AbortController
Target	Cuts P95 / P99 first-word time.
Edge cases	Costs a little extra on slow requests only.

I3 Observability	MVP
What it does	Every stage is timed: capture → STT partial → end-of-turn → LLM first token → render.
How it works	1. One trace per answer; dashboards for P50 / P95 / P99; alerts on P95 above 1.5 s (Doc 1).
Technology	OpenTelemetry (Node + Electron) · Prometheus + Grafana · Sentry (Electron and Node)
Target	100% of answers traced (sampled payloads, never raw audio).

I4 Auto-Update & Distribution	MVP
What it does	Signed installers and silent updates with staged rollout.
How it works	1. Windows code signing; macOS Developer ID + notarisation (start early — lead times are long). 2. Rollout 10% → 50% → 100% with automatic stop on error spikes (Doc 1 canary idea).
Technology	electron-builder · electron-updater · GitHub Actions CI
Target	Update applies on next restart.

I5 Security	MVP
What it does	No provider API keys on the client; short-lived tokens only; encrypted data.
How it works	1. Electron hardening: contextIsolation, sandboxed renderers, no Node in renderers. 2. TLS everywhere; encryption at rest; secrets in a managed secret store.
Technology	Electron security settings · safeStorage · AWS Secrets Manager / Fly secrets
Target	—
Edge cases	Regular dependency audits.

I6 Compatibility Test Harness	v1
What it does	Automated checks across Zoom, Meet, Teams and Webex, window and full-screen share, Windows 10 (19041+) to 11, macOS 13 to latest.
How it works	1. Nightly runs on test machines; re-run after every OS or meeting-app update.
Technology	Scripted test VMs / devices · Playwright for browser-based meeting clients · GitHub Actions
Target	Regressions caught within 24 h.

I7 Latency & Accuracy Bake-off Harness	MVP
What it does	Replays a library of recorded interview audio through STT and LLM providers to compare accuracy, turn timing and first-token time.
How it works	1. Run before choosing vendors and again every quarter.
Technology	Node / Python scripts · Stored audio corpus with reference transcripts
Target	—
Edge cases	Include Indian, US and other accents, cross-talk and jargon.

SECTION 14 · GROWTH

Growth features (later, from the Build Roadmap)

These turn the assistant into a career platform. Build them only after the live assistant has paying users.

Feature	What it does	Technology
Job matching feed	Multi-source job ingestion, deduplication, match score vs profile	Apify / Crawlee, OpenSearch, pgvector
Application tracker	Status pipeline, reminders, resume version per application	Next.js, Postgres, BullMQ
Resume tailoring & auto-apply	JD gap analysis, tailored resume, cover letter, approval controls, audit log	LLM, Playwright, S3
Chrome extension	Autofill, 1-click apply, save job from any page	Chrome Extension MV3
Offer comparison & negotiation coach	Total comp comparison, counter-offer scripts	LLM, Postgres
Skill gap & learning plan	Missing skills for target roles, course suggestions	LLM, pgvector
Mobile app	Alerts, status updates, practice on the go	React Native / Expo
API, webhooks, white-label	Integrations and branded version for colleges / bootcamps	GraphQL / REST, webhooks

SECTION 15 · MATRIX

Technology-to-feature matrix

Technology	Used by features
Rust native module (napi-rs)	B1, B2, B3, D1 (optional)
Electron (BrowserWindow, globalShortcut, safeStorage, desktopCapturer)	A1, C7, D1, E1, E3, E4, E5, I4, I5
React + TypeScript	A2, A3, A5, B3, B5, E1, E2, E5, E6, F3
Local orchestrator (TypeScript rules)	C1, C2, C3, C6, C7
Deepgram Flux	B4, B6, C1, C2, F1
AssemblyAI Universal-Streaming	B4 (backup), B6, I1
Claude Haiku tier / Gemini Flash tier	A2, A3, C4, C6, D3 (approach line)
Claude Sonnet / Opus tier, GPT-5 tier	A4, C5, D3, D4, D5, F1, F2, G2
Streaming TTS	F1
Provider prompt caching	A4, C4, C5, D3
Fastify Edge API	A1, A3, B4, C4, C5, H1, H2, H4, I1, I2
Redis	A4, C6, H2, H4, I1
BullMQ workers	A2, F2, G2, G3, H2
Postgres	A1–A5, F2, F3, G1, G3, H1–H3
S3 / R2	A2, D3, G1, G3
Supabase Auth	A1
Stripe / Razorpay	H1, H4
OpenTelemetry, Prometheus, Grafana, Sentry	H3, I3
GitHub Actions, electron-builder/updater	I4, I6, I7

SECTION 16 · PERFORMANCE

Performance targets

Stage	Target
Audio frame + network to STT	20–60 ms
Early end-of-turn decision	150–250 ms
Local question classification	<10 ms
Network to Edge API → provider (warm)	10–30 ms
LLM time-to-first-token (cached prefix)	300–500 ms
Overlay paint of first token	<16 ms
First word visible (fast path)	**~0.5–0.85 s**
Deep answer first token	~1–2 s (fast answer already visible)
Screenshot solve first word	~0.7–1.6 s

Production SLO	Target	Alert
First word P50 / P95	≤ 0.8 s / ≤ 1.5 s	P95 > 1.5 s
STT partial latency	≤ 300 ms	> 500 ms
False trigger rate	≤ 5%	> 10% daily
Speculation waste	≤ 30%	> 40%
Session failure rate	< 1%	> 1%
Model + infra cost per 45-min session	~$0.30–0.40	> $0.60

SECTION 17 · RELEASE PLAN

What ships when

Stage	Duration	Features	Exit criteria
MVP	~8–12 wks	A1, A2, A3, A4, A5, B1, B2, B3, B4, B5, C1, C3, C4, C6, C7, C8, D1, D2, D3, D5, E1, E2, E3, E6, G3, H2, I3, I4, I5, I7	Live answers on Windows + macOS; first word P50 ≤ 1.1 s; screenshot solve working.
v1	+7–11 wks	B6, C2, C5, D4, E4, E5, G1, G2, H1, H3, H4, I1, I2, I6	First word P50 ≤ 0.8 s; failover tested; compatibility matrix green; billing live.
v1.1	+4–6 wks	B7, F1, F2, F3	Practice mode live; beta feedback ≥ 4/5.
Growth	ongoing	Section 14	Paying users and retention justify expansion.

SECTION 18 · RISKS

Risks & open checks

Risk / check	Action
macOS 15+ privacy-mode gap	State per-OS reliability in-app (E4) and in marketing; watch Apple releases.
Meeting apps change capture method	Nightly compatibility harness (I6); fast patch releases.
Privacy mode never hides the process or defeats cameras / proctoring tools	Say so clearly in the product and terms.
Legal / ethical / platform-terms exposure	Legal review of claims and terms; Practice mode (F) as a lower-risk product line.
Vendor feature assumptions	Verify early end-of-turn events, temporary tokens, keyterms and prompt caching in current docs before building on them.
Model names and prices change fast	Re-check Anthropic (docs.claude.com), OpenAI and Google model lists and prices; re-run the bake-off (I7) each quarter.
Speculation cost	Track waste rate; tune thresholds per round type.
Provider outages	Circuit breakers, failover and hedging (I1, I2); transcript-only fallback.

This specification is built from the two uploaded planning documents and the Unified Low-Latency Blueprint. Library and vendor names are recommendations; all latency numbers are engineering targets, not measurements. No live web research was performed for this document.